

Learning Causal Polytrees *

Juan F. Huete and Luis M. de Campos

Dpto. de Ciencias de la Computacion e I.A.
Universidad de Granada
18071 - Granada, Spain

Abstract. The essence of causality can be identified with a graphical structure representing relevance relationships between variables. In this paper the problem of inferring causal relations from patterns of dependence is considered. We suppose that there exists a causal model, which is representable by a polytree structure and present an approach to the recovering problem. With this approach we can recover efficiently a polytree structure using marginal and conditional independence tests.

1 Introduction

The notion of causality is central to the understanding of human reasoning. Humans, usually, model empirical phenomena as cause-effect relationships. This concept of causality, permits us to deal with reasoning problems efficiently, separating the relevant from the superfluous. In this paper the problem of inferring causal relations from patterns of dependence is considered. In our approach, we suppose that true cause-effect relationships are represented as connected directed acyclic graphs (dags) where each node represents a variable in the domain and the parents of that node correspond to its direct causes. This causal scheme is hidden, but in some way we can test relevance relationships between variables in the domain, trying to recover the causal scheme. Constructing a graph in this manner reflects conditional independence that humans feel exist among causes and effects.

The relevance or irrelevance between two variables is viewed as a test of dependence or independence between the variables. Different formalisms give rise to different definitions of relevance. Note that we don't use any specific formalism to encode the causal schema, so that any formalism could be used.

We can find many works related to the problem of recovering a causal scheme under probabilistic formalism, [1, 2, 9]. In some cases, the relevance relations are obtained from a set of data [5], by a distance measure among variables [4, 8] or directly from experts judgments [11].

In the next section we introduce the previous concepts necessary to define a causal scheme. In section 3 we present some concepts that permit us to develop an algorithm which recover a polytree structure, that is, a singly connected dag, that represents the model. In the last section, some conclusions are presented.

* This work has been supported by the European Economic Community under Project Esprit III b.r.a. 6156 (DRUMS II)

2 Causal schemes

The causal scheme could be considered as a Dependence Model [9], namely a rule that assigns truth values to the predicates " X is independent of Y given Z ", denoted by $I(X \mid Z \mid Y)$, where X, Y , and Z are disjoint subsets of variables in the model. That is, we can test if a subset Z of elements takes part on a relation among the elements in X and those in Y .

We shall restrict our discussion to singly connected causal structures, therefore any pair of nodes is connected via no more than one trail. In that path, we can find a tail to tail node x , $(\alpha \leftarrow x \rightarrow \beta)$ or a tail to head node x , $(\alpha \rightarrow x \rightarrow \beta)$ representing the fact that α and β are dependent variables, but become independent variables given x . On the other hand a head to head node x could be found, $(\alpha \rightarrow x \leftarrow \beta)$, which represents that α and β are independent variables, but become dependent when x is known.

The previous ideas of dependence or independence between nodes in a graphical structure, will be extended with the following definition [6], called d -separation by Pearl [9]

Definition 1. *d-separation:*

1. Let Z be a set of nodes in a dag. A trail t is active by Z if (a) every head to head node with respect to t either is in Z or has a descendent in Z and (b) every other node along t is outside Z . Otherwise, the trail is said to be blocked by Z .
2. Let D be a causal network and X, Z and T three disjoint sets of nodes. Then, X and Y are said to be graphically independent given Z if there exists no active trail by Z between any node in X and any node in Y . Otherwise, X and Y are graphically dependent given Z .

Let M be a causal scheme, we say that a dag is an Independence map (I-map) if every d -separation in the graph implies an Independence in M . On the other hand, a causal network is called a Dependence Map (D-map) if every independence relation in the model, implies a d -separation in the dag. A dag is a *perfect* map of the dependency model if it is both an I-map and a D-map, in that case, we say that the dependence model M is *dag-isomorph*.

3 Recovering Polytrees Structures

To deal with the recovering problem, we only consider those causal schemes M , that can be represented by a simple dag, that is, a structure where between any two nodes, there exists only one path. Following Pearl's notation [9], we call such structures polytrees because they can be viewed as a collection of several causal trees fused together at the nodes where arrows converge head to head $(\rightarrow x \leftarrow)$.

For a node x , we will denote by A_x the set of nodes $y \in M$ such that x and y are dependent (marginally). In a polytree structure T , two variables x, y

are dependent (marginally dependent) if there is no head to head node in the unique path connecting them, otherwise they are marginally independent.

Let x be a variable in M , and y, z two variables in Λ_x . By making Conditional Independence test, we can establish if y lies in the path connecting x and z .

To solve the recovering problem, initially, we consider an empty structure T . This structure reflects, at any moment, the skeleton that could be obtained if we only consider those nodes belonging to T . In each step, a new node will be included in the structure, and making conditional independence tests, the final position of this node in T is found. The next definition helps us in this process.

Definition 2. Let T be a causal structure, and let x be any variable in T . A Sheaf structure for x , denoted by Ψ_x , is the set of variables in the structure which are connected directly to x ; therefore, a Sheaf structure will include the direct causes and the direct effects of x in T .

The next theorem permits us to detect whether the inclusion of a new variable in the structure T will affect a Sheaf structure for a node x in T .

Theorem 3. Let M be a causal scheme, let x be any variable in M represented by a node in T , and Ψ_x be a Sheaf structure for x . Let z be a variable in M , but not in T and z belonging to Λ_x . After including z in T , Ψ_x will be modified if and only if one of the following conditions holds

1. $I(x | z | y) = \text{True}$ in M for some $y \in \Psi_x$.
2. $I(x | y | z) = \text{False}$ in M , for all $y \in \Psi_x$.

If the previous theorem does not hold, then the node z does not belong to the Sheaf structure for x . The following theorem gives us a method which, directing the search through the partial structure, permits to detect those variables whose Sheaf structure must be modified.

Theorem 4. In the same conditions of the previous theorem, if the conditions of this theorem do not hold, then there exists one and only one node $y \in \Psi_x$ such that $I(x | y | z) = \text{True}$ in M .

Theorem 5. The conditions in Theorem 3 and Theorem 4 are not satisfied simultaneously.

Now, we will see graphically, how a Sheaf structure for a node x will be affected when we insert a new variable z , with $z \in \Lambda_x$, in the structure T . Suppose that for a node x in T , the figure 1.a) represent the Sheaf structure for x . Firstly, when theorem 3 holds we know that the Sheaf structure for the node x will be modified by the inclusion of z in T . In this case, if condition one is verified, the set of nodes belonging to Ψ_x is divided in two subsets, one which includes those nodes y_i , with $i = 1, \dots, j$ where $I(x | z | y_i)$ is true, and the other which includes those nodes v_k , with $k = 1, \dots, l$ where $I(x | z | v_k)$ is false. For any variable $y_i \in \Psi_x$ such that $I(x | z | y_i)$ holds, we must add two

new edges, one between x and z and the other between z and y_i , and remove the edge between x and y_i . With these changes, z is now in Ψ_x and the nodes $y_i \notin \Psi_x$. This represents the fact that if z is known the variables x and y_i become independent, (see fig. 1b). If the condition two from Theorem 3 is verified, and condition one does not hold, add a new edge between x and z which reflects the knowledge that x and z are related variables (remember that z belong to Λ_x) and none of the nodes in Ψ_x affects this relation. (see fig. 2b)

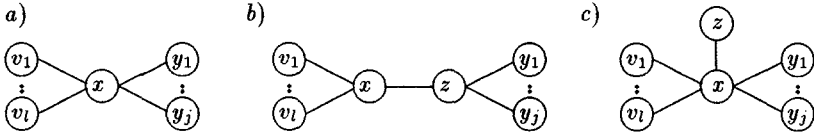


Fig. 1. a) Sheaf structure for x . b) Theorem 3.1 c) Theorem 3.2

When Theorem 4 holds, let $\Psi_x = \{v_1, \dots, v_l, y\}$ and let y be the variable that makes true the relation $I(x \mid y \mid z)$. Then we know that y is in the path connecting x and z , and therefore our original problem can be solved by studying whether z belongs to the Sheaf structure for y (Ψ_y). (see fig. 2)

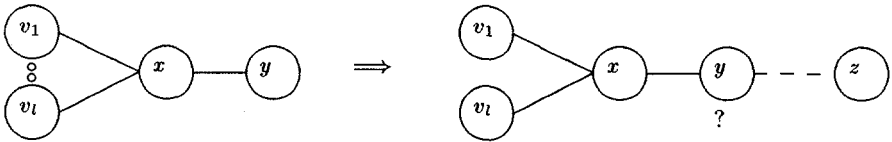


Fig. 2. Theorem 4 holds

As we said, our algorithm starts with an empty structure T , and then a variable x is selected, finding those variables that are dependent with x , Λ_x . Then, we construct a structure T which includes x and the variables in Λ_x . In the following steps, select a new variable y from the structure T , find Λ_y , and for any $z \in \Lambda_y$ such that $z \notin T$ try to insert z in T . Note that, in the path between y and z , there are no head to head nodes; then, we could apply the previous theorems. Repeat this step until all the variables are in T .

The algorithm constructs a polytree structure representing a causal scheme in $O(n^2)$ steps, with n representing the number of variables in the scheme. Note that we only make marginal and first order conditional independence tests. With this algorithm, we can construct efficiently the skeleton of a polytree, but we do not mention how to recover the directionality of the arcs. We only could recover

Algorithm

1. For each variable x in M Initialize Λ_x
 $Visited[x]=False$; $Expand[x]=False$
 For each variable y in M , such $x \neq y$
 If $I(x, y)=False$ Then Add y to Λ_x
2. Select a node x from M , assign x to T ; $Expand[x]=True$
3. While there exist non visited nodes in T
 - (a) Select a non visited node x from T ; $Visited[x]=True$
 - (b) While there exist non expanded nodes in Λ_x , select a non expanded node z from Λ_x , new node to insert in T
 - i. $Expand[z]=True$
 - ii. $Forward=True$
 - iii. $Inserted=False$
 - iv. While $Forward=True$ do
 If all $y \in \Psi_x$ are tested
 Then $Forward=False$
 Else
 Select a new y in Ψ_x
 If $(x \mid y \mid z)=True$
 Then $x = y$
 - v. For every y in Ψ_x do
 If $(x \mid z \mid y)=True$
 Add (x, z) and (z, y) to T
 Delete (x, y) from T
 $Inserted=True$
 - vi. If $Inserted=False$ Then Add (x, z) to T
 - vii. Reset x to the original value

some oriented arcs, those where a head to head node is present, but we are free to choose the orientation of some set of nodes, those where a path $(x \leftarrow z \leftarrow y)$, $(x \leftarrow z \rightarrow y)$ or $(x \rightarrow z \rightarrow y)$ is present, as long as we do not introduce head to head patterns. These three orientations are indistinguishable, as they represent the same independence relationships. To recover the head to head orientation $(x \rightarrow z \leftarrow y)$ we could use the following test: " If two variables x and y are marginally independent and, given z , they become conditionally dependent, then the structure is oriented with a head to head arc".

4 Concluding Remarks

Learning algorithms, usually, need high order independence tests involving a great number of variables, such as $I(x \mid U - x, y \mid y)$, with U representing the set of variables in the causal scheme, and use a exponential number of independence relations. There are some algorithms [6, 7] which are polynomial in

the number of independence verifications that recover simple structures. Chow and Liu [4] proposed a different approach to recover tree structures, in which a dependence measure among variables is used. The algorithm works under probabilistic formalism, recovering a tree structure in $O(n^2)$ steps and was extended by Rebane and Pearl [10] to recover polytrees. In this paper a learning algorithm which recover simple structures, in order $O(n^2)$, such as trees and polytrees, is given. The input to the algorithm can be obtained either from empirical data or from expert judgments. In future works we will try to extend the algorithm to structures where some kind of cycles are allowed, such as simple dags.

On the other hand, we are interested in learning causal structures from a dependence model where independence relationships are based on any formalism for uncertainty, as probability theory, Dempster-Shafer's theory, upper and lower probabilities,...[3]. With these formalisms, testing for independence relationships may be expensive, so we are interested in doing only first order independence tests.

References

1. Acid S., Campos L.M. de, González A. Molina R., Pérez de la Blanca N.: Learning with CASTLE, in Symbolic and Quantitative approaches to Uncertainty. Lecture Notes in Computer Science 584, R. Kruse R., Siegel P. (Eds). Springer Verlag, (1991) 99-106.
2. Acid S., Campos L.M. de, González A. Molina R., Pérez de la Blanca N.: CASTLE: A tool for bayesian learning. Proceedings of the ESPRIT 91 Conference, Commission of the European Communities, (1991) 363-377.
3. Campos L.M.de, Huete J.: Independence concepts in Upper and Lower probabilities. Proceedings of the Fourth IPMU Conference, (1992) 129-132.
4. Chow C.K., Liu C.N.: Approximating discrete probability distribution with dependence trees. IEEE Transactions on Information Theory 14, (1968) 462-467.
5. Cooper G., Herskovits E.: A bayesian method for the induction of Probabilistic Networks from data. Machine Learning 9, (1992) 309-347.
6. Geiger D., Paz A., Pearl J.: Learning causal trees from Dependence information. Proceedings of the Eighth National Conference on A.I. (1990) 770-776.
7. Geiger D., Paz A., Pearl J.: Learning Simple Causal Structures International Journal of Intelligent Systems, Vol 8, (1993) 231-247.
8. Herskovits E., Cooper G.: KUTATO: An Entropy-Driven system for construction of probabilistic expert systems from Databases. Proceedings of the 6th Conference on Uncertainty in Artificial Intelligence, Cambridge MA. (1990) 54-62.
9. Pearl J.: Probabilistic Reasoning in intelligent systems: Networks of plausible inference. Morgan and Kaufman, San Mateo (1988).
10. Rebane G., Pearl J.: The recovery of causal poly-trees from statistical data, Uncertainty in Artificial Intelligence 3, Kanal L.N., Levitt T.S. and Lemmer J.F. Eds. North-Holland. (1989) 175-182.
11. Srinivas S., Russell S., Agogino A.: Automated construction of sparse Bayesian networks from unstructured probabilistic models and domain information. Uncertainty in Artificial Intelligence 5, Henrion M., Shachter R.D., Kanal L.N., Lemmer J.F. (Eds), North-Holland, Amsterdam. (1990) 295-308.